BARUCH COLLEGE

Department of Mathematics • Master of Financial Engineering

DEEP LEARNING IN FINANCE | Lecture Series

Theoretical Foundations of Deep Learning

Parts I–III — Approximation, Generalization & Complexity

Oualid Missaoui

Adjunct Faculty, Baruch College

Overview

Part I — Approximation & Scaling

1. **Why deep learning works**
2. **The learning problem**
3. **Approximation**
4. **Neural scaling laws**

Part II — Generalization & Spectra

1. **Classical statistical learning theory**
2. **Double descent**
3. **Double descent in linear regression**
4. **Implicit bias of gradient descent**
5. **Linearizing neural networks: the NTK**
6. **Spectral theory of generalization**

Part III — The Virtue of Complexity

1. **The claim**
2. **What is actually traded**
3. **The evidence**
4. **The lesson**

The Part II phenomena, in asset pricing: does complexity pay out of sample?

PART I

Approximation & Scaling Laws

Why deep learning works: the learning problem, universal approximation, depth, and neural scaling laws.

Part I — Agenda

Why deep learning works

The learning problem

- Bias, variance & the error decomposition
- Double descent

Approximation

- Universal approximation
- Complexity & the curse of dimensionality
- Compositionality & depth

Neural scaling laws

- Power laws
- Compute-optimal training
- Practice & limits

Roadmap

Why deep learning works

The learning problem

- Bias, variance & the error decomposition
- Double descent

Approximation

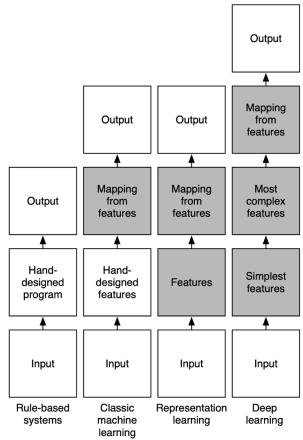
- Universal approximation
- Complexity & the curse of dimensionality
- Compositionality & depth

Neural scaling laws

- Power laws
- Compute-optimal training
- Practice & limits

What is deep learning?

- **Big idea:** Deep learning learns a *hierarchy of representations* directly from raw data and optimizes the whole pipeline end-to-end.
- **Why it matters:**
  - Replaces manual feature engineering with learned features at multiple levels of abstraction.
  - Scales with data + compute; performance keeps improving as both grow.
  - Encodes powerful inductive biases (e.g., parameter sharing in convs, attention in Transformers, compositionality).
  - Enables transfer: pretrain on broad data $\rightarrow$ fine-tune for a task.



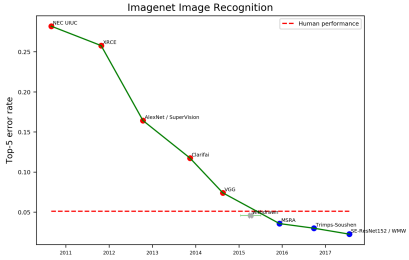
Source: Y. Bengio, *Deep Learning* (MLSS Austin).

The 2012 vision breakthrough

Who's who

NEC UIUC — NEC Labs America + U. Illinois (2010)
XRCE — Xerox Research Centre Europe (2011)
AlexNet / SuperVision — Krizhevsky, Sutskever & Hinton, U. Toronto (2012)
Clarifai — M. Zeiler's start-up, "ZFNet" (2013)
VGG — Visual Geometry Group, Oxford (2014; GoogLeNet won)
Withdrawn — Baidu (2015), disqualified for breaking submission rules
MSRA — Microsoft Research Asia — ResNet (2015)
Trimps-Soushen — research institute of China's Ministry of Public Security (2016)
SE-ResNet152 / WMW — Squeeze-and-Excitation nets, team WMW (2017)

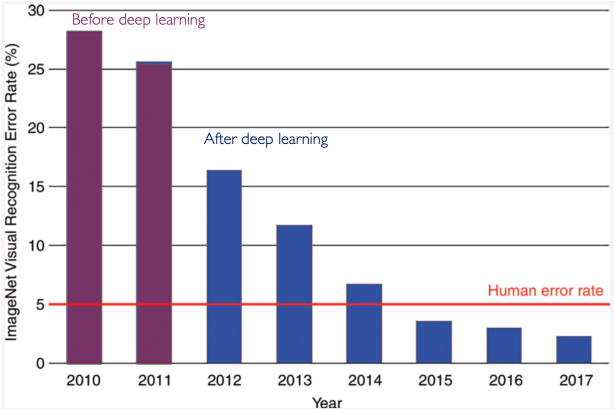
Top-5 error: the true label is not among the model's 5 best guesses. Human ≈5% (Karpathy, 2014).



Source: EFF, AI Progress Measurement (ImageNet)

- **2012 — AlexNet**: first deep net to win ImageNet; top-5 error $\sim$26% $\rightarrow$ $\sim$16% in one year.
- Then a deeper model almost every year — VGG, GoogLeNet, ResNet (MSRA).
- Crosses the $\sim$**5% human line ($\approx$2015)**; $\sim$2–3% by 2017.
- **Lesson:** learned representations beat decades of hand-crafted vision features.

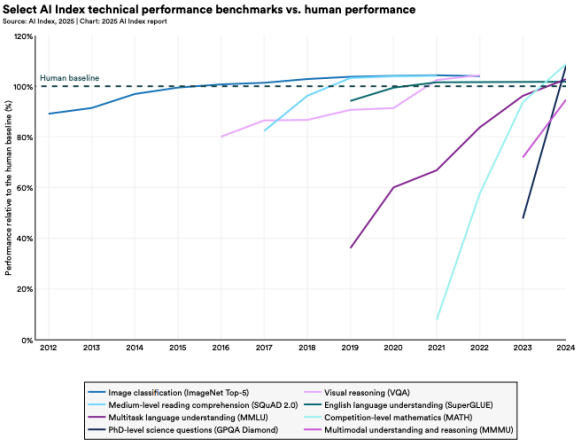
Vision: a regime change, not a tweak



Source: NeurIPS 2022 talk slides

- The bars split history in two: **before** deep learning (2010–11) vs **after** (2012+).
- Hand-engineered pipelines had stalled near $\sim$**26–28% error**.
- Deep CNNs trained end-to-end broke the plateau and passed **human-level accuracy**.
- **Takeaway:** the gain came from *representation learning*, not better feature engineering.

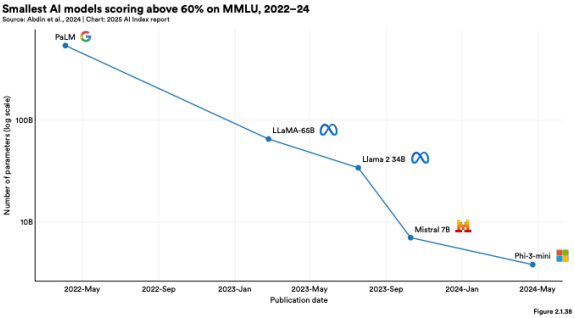
The recipe generalizes beyond vision



Source: Stanford HAI, AI Index Report 2025, Fig. 2.1.33

- The **same recipe generalized** far beyond vision.
- Reading (SQuAD), language (SuperGLUE, MMLU), reasoning (GPQA, MATH), multimodal (VQA, MMMU).
- Most benchmarks now **reach or pass the human baseline** (dashed line).
- Competition-math & PhD-science still lag — **today's frontier**.

…and it gets cheaper



Source: Stanford HAI, AI Index Report 2025, Fig. 2.1.38

MMLU = *Massive Multitask Language Understanding* — a 57-subject multiple-choice exam; random guessing scores 25%.

- **Cost collapsed, capability rose:** the same exam score now needs a far smaller model.
- Reaching $>$**60% MMLU** fell from **540B params (PaLM, 2022)**…
- …to $<$**4B (Phi-3-mini, 2024)**: $\sim$**100**$\times$ **fewer parameters** in two years.
- **Lesson:** capability broadened *and* got cheaper $\Rightarrow$ the bottleneck shifts to “reliable, cheap, verifiable?”

From Breakthrough to Deployment (2024–25)

- **Conclusions**
  - Error collapsed: perception tasks surpassed human level; gains have plateaued on classic benchmarks.
  - Capability broadened: strong progress across vision, language, multimodal; uneven on math/logic.
  - Efficiency jumped: “good-enough” performance now with small models ($<$10B), slashing cost/latency.
- **Implications**
  - New bottleneck = reasoning, reliability, and transfer; less about raw recognition.
  - Best ROI: domain-tuned small models + tool use + retrieval/knowledge graphs.
  - Eval must shift to cost-quality-safety: robustness, data provenance, and verifiability.

Roadmap

Why deep learning works

The learning problem

- Bias, variance & the error decomposition
- Double descent

Approximation

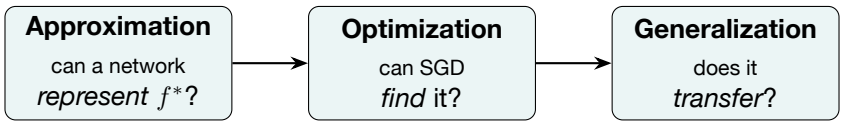
- Universal approximation
- Complexity & the curse of dimensionality
- Compositionality & depth

Neural scaling laws

- Power laws
- Compute-optimal training
- Practice & limits

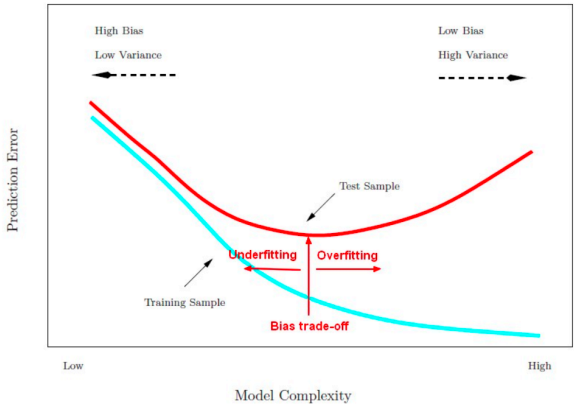
Three questions about any learned model

Deep learning *works* — but to reason about *why*, separate three questions:



- They map onto the error decomposition: **approximation** $\leftrightarrow$ *bias*; **optimization** $+$ **generalization** $\leftrightarrow$ the *variance* / estimation gap.
- **Part 1:** approximation (what nets *can* represent) $+$ the empirical **scaling laws** for how error falls with resources.
- **Part 2:** optimization & generalization — why SGD finds minima that transfer.

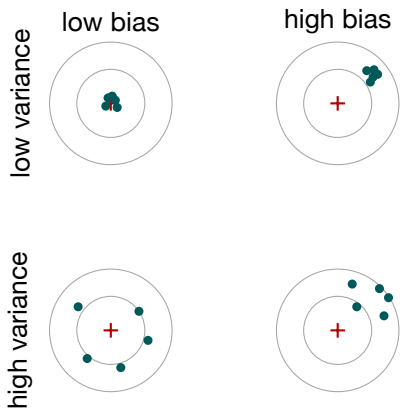
Classic learning theory: the bias–variance trade-off



Source: Hastie, Tibshirani & Friedman, *The Elements of Statistical Learning*, Fig. 2.11

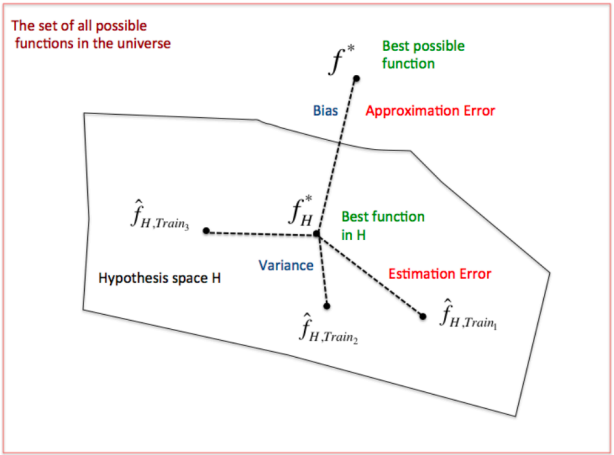
- As model complexity grows, **training error** keeps falling but **test error** is **U-shaped**.
- Left = **underfitting** (high bias, low variance); right = **overfitting** (low bias, high variance).
- The train–test gap widens with complexity — that gap is *variance*.
- Goal: sit at the **sweet spot**, or push the whole curve down with more data / regularization.

Bias and variance, at a glance



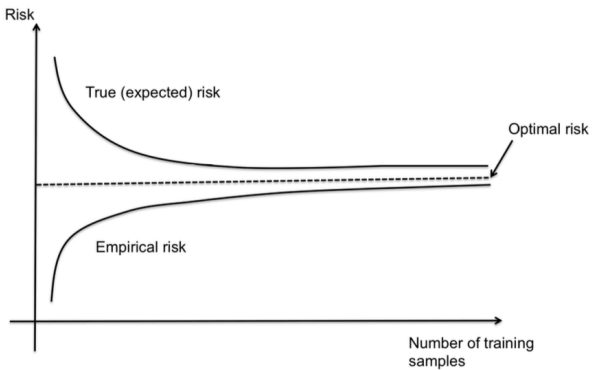
- **Bias** $=$ how far the shots sit *from the bullseye* on average (model class too simple / mis-specified).
- **Variance** $=$ how *scattered* the shots are across training sets (too sensitive to the particular sample).
- Each dart $=$ a model fit on a *fresh* training set; the bullseye $= f^*$.
- Test error combines **both** (plus irreducible noise) — the trade-off is choosing where to sit in this square.

Where does the error come from? Approximation vs estimation



- f^* = best possible function; H = functions your model *can* represent.
- **Approximation error (bias):** $f^* \to f_H^*$ — how far the best-in-H sits from the truth.
- **Estimation error (variance):** $f_H^* \to \hat{f}_{H,\text{Train}}$ — finite data lands you on a different function each draw.
- **Test risk** = irreducible noise + approximation + estimation.

Learning curves: more data shrinks estimation error

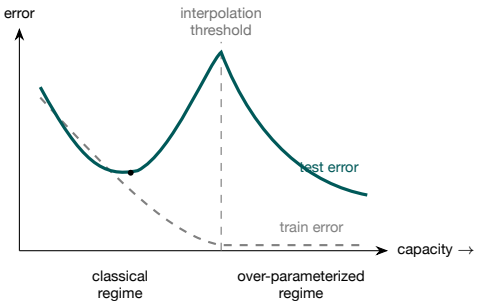


- With few samples the **empirical risk** underestimates and the **true risk** is high — a large gap.
- As n grows, both converge toward the **optimal risk** for a fixed hypothesis space H.
- That closing gap *is* the **estimation error** — more data shrinks it.
- But bias (approximation error) does *not* vanish with data — only a richer H reduces it.

Bias–Variance: what to remember

- Test risk combines irreducible noise, approximation error, and estimation error.
- Increasing model capacity usually lowers approximation error but raises estimation error unless you add data or regularization.
- Underfitting appears when both training and validation errors remain high.
- Overfitting appears when training error is low but validation error is high.
- When both errors are high — noisy labels or poor targets — fix the data first.
- You control three dials: model capacity, regularization, and the quality and quantity of data.
- Practical regularization includes weight decay, dropout, data augmentation, and early stopping.
- In deep learning, optimization provides implicit regularization and can produce double descent, yet the bias–variance trade-off still governs generalization.
- Plot learning and capacity curves first so you know whether to buy data, add regularization, or scale the model.
- Aim for the best outcome per dollar: compact, well-regularized models on high-quality data.

The modern twist: double descent



- Classical U-curve: below the **interpolation threshold** (training error $= 0$), bias falls, then variance rises.
- Push *past* it — keep adding parameters — and test error **descends again**, often below the classical best.
- Modern nets live in this **over-parameterized** regime, which bias–variance alone does not explain.
- *Why* it descends twice (implicit bias of SGD, flat minima) is a central question of **Part 2**.

Roadmap

Why deep learning works

The learning problem

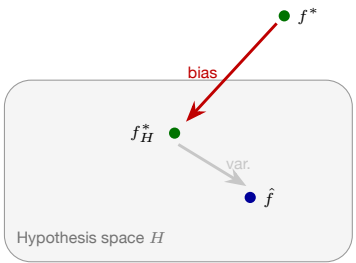
- Bias, variance & the error decomposition
- Double descent

Approximation

- Universal approximation
- Complexity & the curse of dimensionality
- Compositionality & depth

Neural scaling laws

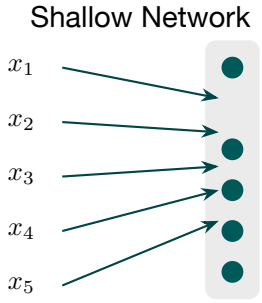
- Power laws
- Compute-optimal training
- Practice & limits



The Universal Approximation Theorem

Theorem. Let $g(x)$ be a continuous function defined in a compact subset $S \subset \mathbb{R}^l$ and any $\epsilon > 0$. Then there is a two layer neural network $\hat{g}(x)$ with $K(\epsilon)$ hidden nodes so that

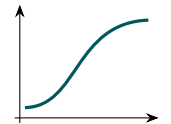
$$|g(x) - \hat{g}(x)| < \epsilon \quad \forall x \in S$$



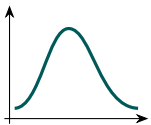
- A single hidden-layer network with a non-linear activation can approximate any continuous function on a bounded input domain.
- *Compact subset* just means we restrict inputs to a normalized, bounded region; exactly what we do in practice.
- The result is about representational capacity only; it does not guarantee that training will find the best weights or need only a little data.

George Cybenko. Approximation by superpositions of a sigmoidal function. Mathematics of control, signals and systems, 2(4):303-314, 1989.

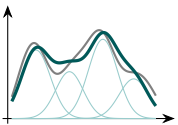
Why it's true: a network builds f from sigmoids



one sigmoid $\sigma(wx+b)$



two make a bump



sum of bumps $\approx f$

- A single unit $\sigma(wx+b)$ is a soft **step**: w sets steepness, b sets position.
- **Two opposing steps make a localized bump** of adjustable height and width.
- Enough bumps **tile** any continuous f to accuracy ε — that *is* the UAT.
- The catch (next slides): the proof is **non-constructive** — it shows such weights *exist*, not how to find them.

Proof of the Universal Approximation Theorem (1)

Definition. A function σ is *discriminatory* if given a measure $\mu \in M(I_n)$ such that

$$\int_{I_n} \sigma(w^T x + b)\, d\mu(x) = 0,\ \forall w \in \mathbb{R}^n, b \in \mathbb{R}$$

implies that $\mu = 0$.

Definition. A function $\sigma : \mathbb{R} \to \mathbb{R}$ is called *sigmoidal* if it is non-decreasing and satisfies the asymptotic conditions

$$\lim_{z \to -\infty} \sigma(z) = 0 \quad \text{and} \quad \lim_{z \to \infty} \sigma(z) = 1.$$

An example is the logistic function $\sigma(z) = \frac{1}{1+e^{-z}}$.

Lemma. Any bounded, measurable, sigmoidal function is discriminatory. In particular, any continuous sigmoidal function is discriminatory.

Proof of the Universal Approximation Theorem (2)

Proof. Let $C(S)$ denote the space of continuous functions on the compact set $S \subset \mathbb{R}^n$, equipped with the uniform norm. Define the set of two-layer neural networks:

$$\mathcal{N} = \left\{ \hat{g}(x) = \sum_{k=1}^{K} \alpha_k \sigma(\beta_k^T x + \gamma_k) \;\middle|\; \alpha_k, \gamma_k \in \mathbb{R},\ \beta_k \in \mathbb{R}^n,\ K \in \mathbb{N} \right\},$$

where $\sigma : \mathbb{R} \to \mathbb{R}$ is a continuous, sigmoidal activation function. We will show that $\mathcal{N}$ is dense in $C(S)$.

Suppose, for contradiction, that $\mathcal{N}$ is **not dense** in $C(S)$. By the **Hahn–Banach theorem**, **there exists** a non-zero bounded linear functional $L : C(S) \to \mathbb{R}$ that vanishes on $\mathcal{N}$, i.e., $L(f) = 0$ for all $f \in \mathcal{N}$.

By the **Riesz representation theorem**, any bounded linear functional on $C(S)$ **can be represented by** a finite signed Borel measure μ on S:

$$L(f) = \int_S f(x)\, d\mu(x) \quad \forall f \in C(S).$$

Since L vanishes on $\mathcal{N}$, we have:

$$\int_S \sigma(\beta^T x + \gamma)\, d\mu(x) = 0 \quad \forall \beta \in \mathbb{R}^n,\ \gamma \in \mathbb{R}.$$

Highlighted = existence-only steps: they assert something *is there*, never how to build it.

Proof of the Universal Approximation Theorem (3)

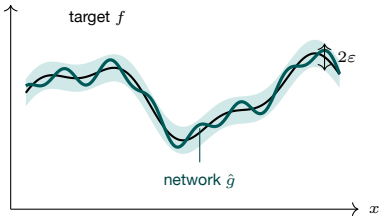
By the lemma established above, any continuous sigmoidal function σ is discriminatory. Therefore, the integral condition $\int_S \sigma(\beta^T x + \gamma)\, d\mu(x) = 0$ implies $\mu = 0$, which contradicts the assumption that L is a non-zero functional.
Hence, $\mathcal{N}$ must be dense in $C(S)$. Density implies that for any $g \in C(S)$ and $\epsilon > 0$, there exists $\hat{g} \in \mathcal{N}$ such that:

$$\sup_{x \in S} |g(x) - \hat{g}(x)| < \epsilon.$$

In particular, $\hat{g}$ can be written as $\hat{g}(x) = \sum_{k=1}^{K(\epsilon)} \alpha_k \sigma(\beta_k^T x + \gamma_k)$, where $K(\epsilon)$ is finite. This completes the proof. □

The proof does **not** help us construct such a shallow network — it only guarantees one exists.

Approximation costs units: the question



- **Accuracy ε.** We want $\hat{g}$ within ε of f *everywhere*: $|f(x) - \hat{g}(x)| \leq \varepsilon$. A smaller ε is a *tighter tube*.
- **Budget N.** A shallow network is a sum of N units ("bumps," as on the UAT slide). N is the **cost** — the width, $\approx$ the parameter count.
- **The question.** UAT promises *some* N works. But how fast must N grow as we shrink ε — and how does that depend on the *target*?

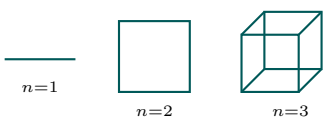
The target's two difficulty knobs: n and m

We now want the **approximation rate**: how fast must the budget N grow as we shrink ε? The answer depends on two properties of the target f.

ε — accuracy · N — network width (the cost) · n — input dimension · m — smoothness

Capital N is what we pay; lower-case n is a property of the problem.

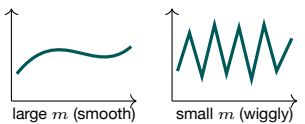
n — input dimension



how many variables f depends on

more dims = more space to cover

m — smoothness



how many derivatives f has

smoother = fewer pieces needed

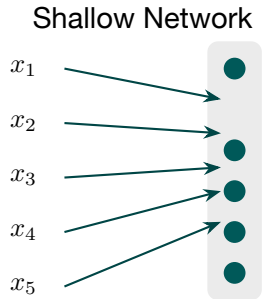
The rate will combine them — **harder with n, easier with m** — and they meet as the ratio n/m. *A theorem says exactly how.*

Complexity of approximation: the theorem

Theorem. Let $\sigma : \mathbb{R} \to \mathbb{R}$ be infinitely differentiable, and not a polynomial. For $f \in W_m^n$ the complexity of shallow networks that provide accuracy at least ϵ is

$$N = \mathcal{O}\left(\epsilon^{-\frac{n}{m}}\right)$$

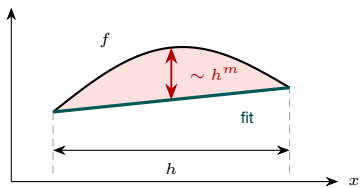
and is the best possible.



And it is *tight*: no shallow architecture beats $\varepsilon^{-n/m}$ over all of W_m^n (m well-behaved derivatives on $\mathbb{R}^n$). **Where does n/m come from?** The next two slides build the count by hand.

Mhaskar & Poggio (2016), Deep vs. Shallow Networks: an Approximation Theory Perspective

Where $\varepsilon^{-n/m}$ comes from (1/2): why a patch is $\varepsilon^{1/m}$ wide



one patch, drawn for $m = 2$: a straight-line fit

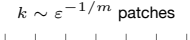
- **Zoom in on one patch** of width h. Over so short a span f barely bends, and Taylor says a degree-$(m-1)$ polynomial tracks it to within $\sim h^m$.
- **Choose h to buy accuracy ε:**

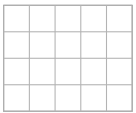
$$h^m \approx \varepsilon \implies h \approx \varepsilon^{1/m}$$

This is the only place m enters the argument.

- **Each patch costs $O(1)$ units.** A bounded number of units reproduces one local polynomial — so the width N is proportional to the *number of patches*.

Where $\varepsilon^{-n/m}$ comes from (2/2): counting patches





$k \times k = k^2$ boxes

$\ldots$ n axes $\Rightarrow$ $k^n = \varepsilon^{-n/m}$ boxes

- **1-D.** Each patch is $h \approx \varepsilon^{1/m}$ wide, so covering $[0,1]$ takes $k = 1/h \approx \varepsilon^{-1/m}$ patches.
- n**-D.** Repeat along *every* axis: $k^n = \left(\varepsilon^{-1/m}\right)^n = \varepsilon^{-n/m}$ boxes — still $O(1)$ units each.
- **So** $N \sim \varepsilon^{-n/m}$: the dimension n lands in the *exponent* (the **curse**); smoothness m divides it (the relief).

What the rate $\epsilon^{-n/m}$ tells us

- **In ε — a power law.** Halving the error multiplies the width by $2^{n/m}$: at $n = 10,\ m = 2$, each factor-of-two in accuracy costs $2^5 = 32\times$ more units.
- **In n — exponential, and this is the wall.** Each extra input variable multiplies the width by $\varepsilon^{-1/m}$. At $\varepsilon = 1\%$, $m = 2$ that is $10\times$ *per dimension*:

dimension n	2	5	10	20
units N	10^2	10^5	10^{10}	10^{20}

  Ten modestly-smooth features already demand ten billion units — not a constant factor, a wall.
- **The relief.** If f is smooth enough that $m = n$, the exponent collapses to ε^{-1}: ≈ 100 units at *any* dimension. The curse is about the *ratio* n/m, not n alone.
- **Tightness.** $\left(\frac{c}{\varepsilon}\right)^{n/m} \le N(\varepsilon) \le \left(\frac{C}{\varepsilon}\right)^{n/m}$ — the exponent cannot be improved. $N =$ width; parameter count $\propto N$, and training/sample requirements grow with N.
- **When it can be beaten.** Extra *structure* — compositionality, low-dimensional manifolds, Barron class — or *depth* can drop the effective n in the exponent. **That is the next idea.**

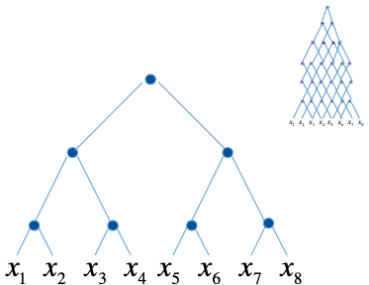
Mhaskar & Poggio (2016), Deep vs. Shallow Networks: an Approximation Theory Perspective

Compositional functions

Formal definition (binary case). Let $k = 2$. For $L \in \mathbb{N}$ and $n = 2^L$, define the class $\mathcal{C}_{2,L}$ of compositional functions $f : \mathbb{R}^n \to \mathbb{R}$ with a binary-tree computational graph:

$$f(x_1, \ldots, x_{2^L}) = h_L\Big(h_{L-1,1}\big(h_{L-2,1}(x_1, x_2), h_{L-2,2}(x_3, x_4)\big),\ h_{L-1,2}\big(h_{L-2,3}(x_5, x_6), h_{L-2,4}(x_7, x_8)\big)\Big)$$

where each constituent map $h_{\ell,j} : \mathbb{R}^2 \to \mathbb{R}$ belongs to W_m^2 (two-variable Sobolev class of order m), and **Depth** $= L = \log_2 n$.



A binary hierarchical network in 8 variables.

Source: Mhaskar & Poggio (2016), Deep vs. Shallow Networks

Why compositional structure matches real data

Real-world signals are built hierarchically — coarse from fine:

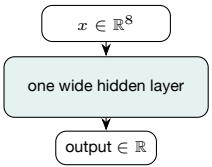
- **Images:** local edges $\rightarrow$ motifs $\rightarrow$ objects (the CNN hierarchy).
- **Time series:** short-range dynamics $\rightarrow$ multi-scale patterns (stacked / dilated convolutions).
- **Tabular:** feature pairs / crosses $\rightarrow$ segment-level aggregations.

Practical benefits: locality & reuse, parameter sharing, better sample efficiency, and a natural inductive bias.

Takeaway: depth aligns the network with the computation graph of f. When f is compositional, we replace one hard n-D approximation by many easy 2-D ones — shrinking the ε-exponent from n to 2.

One hard n-D problem, or many easy 2-D ones?

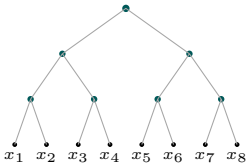
Shallow



fits the *whole* 8-D function at once

$\Rightarrow\ \varepsilon^{-n/m}$ **(curve)**

Deep / compositional



each node fits an easy 2-D map

$\Rightarrow\ (n-1)\,\varepsilon^{-2/m}$ **(linear in n)**

Compositional structure turns **one hard n-D approximation** into **many easy 2-D ones** — the source of depth's efficiency.

Depth Efficiency

Theorem. For $f \in W_m^{n,2}$ consider a *deep network with the same compositional architecture* and with an activation function $\sigma : \mathbb{R} \to \mathbb{R}$ which is infinitely differentiable, and not a polynomial. The complexity of the network to provide approximation with accuracy at least ϵ is:

$$N = \mathcal{O}\left((n-1)\,\epsilon^{-\frac{2}{m}}\right)$$

Deep networks aligned with the compositional structure of the target approximate *much* more efficiently: complexity scales only **linearly in the dimension** n and **polynomially in accuracy**. Compare the shallow rate $\varepsilon^{-n/m}$ — the n has left the exponent.

Telgarsky (2016), Benefits of depth; Mhaskar & Poggio (2016); Poggio et al. (2019), Theoretical Issues in Deep Networks

Shallow vs. deep: the bottom line

Feature	Shallow Networks	Deep Networks
Function class	$f \in W_m^n$ (general Sobolev)	$f \in W_m^{n,2}$ (compositional structure)
Approximation complexity	$\mathcal{O}(\epsilon^{-n/m})$	$\mathcal{O}((n-1)\epsilon^{-2/m})$
Curse of dimensionality	Yes (exponential in n)	Avoided (linear in n)
Optimality	Yes (best possible)	Not necessarily, but much better
Assumes structure in f	No	Yes (hierarchical/compositional)

Takeaway. Depth pays off *only* when the target has compositional structure — then it converts one hard n-D approximation into many easy 2-D ones.

Architecture x Function class comparison

Width $N(\varepsilon)$ needed for error $\leq \varepsilon$

Architecture	**General Sobolev** W_m^n	**Compositional** (binary; $h \in W_m^2$)
Shallow (1 hidden layer)	$\Theta(\varepsilon^{-n/m})$	$\Theta(\varepsilon^{-n/m})$
Deep compositional (depth $\log_2 n$)	Unknown	$\mathcal{O}(n\,\varepsilon^{-2/m})$

- "Unknown" = no established general improvement theorem for a compositional **architecture** when f is an arbitrary W_m^n function (worst-case curse $\Theta(\varepsilon^{-n/m})$ persists for general deep classes).
- Rates hide constants/log factors and depend on the domain/norm.
- For a k-ary compositional graph, replace 2 by k: $\mathcal{O}(n\,\varepsilon^{-k/m})$.

What Universal Approximation Theorems Really Tell Us?

Advantages:

They guarantee representational capacity.

Limitations:

- They do **not** explain why or how deep nets generalize.
- They do **not** address training dynamics or sample efficiency.

- So if representational power isn't enough, what explains generalization in overparameterized deep networks?
- **That is what we will explore in** Optimization & Generalization in the second part of the lecture.

Roadmap

Why deep learning works

The learning problem

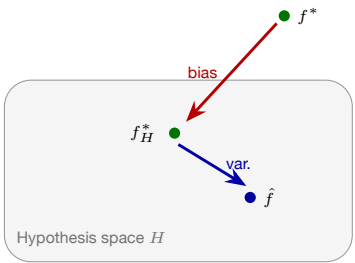
- Bias, variance & the error decomposition
- Double descent

Approximation

- Universal approximation
- Complexity & the curse of dimensionality
- Compositionality & depth

Neural scaling laws

- Power laws
- Compute-optimal training
- Practice & limits



Beyond approximation: empirical scaling laws

- **The hook.** A large model's loss can be **forecast before it is trained** — from pilot runs 100–$1000\times$ smaller. Scaling laws are a *forecasting and budgeting* tool, not a curiosity.
- **Where we are.** Approximation theory said deep nets *can* represent structured functions cheaply — a statement about *capacity*, silent on training and data.
- **The gap.** Real-world success outran those bounds. The practical question is no longer "can it?" but "how does performance *improve as we scale*?"
- **Neural scaling laws.** Empirically, test loss falls as a **power law** in model size N, data D, or compute C — and it does so *predictably*. That is the topic of this module.

The atom: a single learning curve

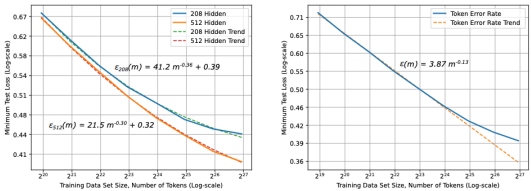


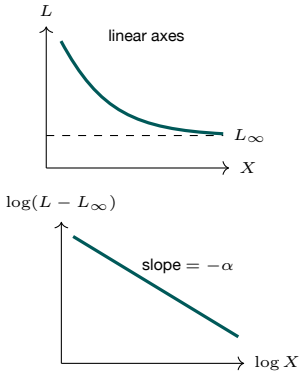
Figure 1: Neural machine translation learning curves. Left: the learning curves for separate models follow $\varepsilon(m) = \alpha m^{\beta_g} + \gamma$. Right: composite learning curve of best-fit model at each data set size.

Neural machine translation: minimum test loss vs. training-set size.

Source: Hestness et al. (2017), Deep Learning Scaling is Predictable

- Hold everything else abundant; vary *one* resource (here, the data D).
- Loss falls **smoothly and predictably** — and on **log–log axes it becomes a straight line**.
- A straight line on log–log ⇔ a **power law**. That single observation is the whole foundation of what follows.

Anatomy of a power law — and how to read one



$$L(X) = \underbrace{L_\infty}_{\text{irreducible}} + \underbrace{c\,X^{-\alpha}}_{\text{reducible}}$$

- L_∞**:** the noise floor / task entropy — no amount of X removes it. $c\,X^{-\alpha}$**:** vanishes as $X \to \infty$.
- **To fit it:** subtract the floor and take logs, $\log(L - L_\infty) = \log c - \alpha \log X$ — a line of slope $-\alpha$, intercept $\log c$. Same recipe on *any* axis (N, D or C).
- **Elasticity:** $\partial \log(L - L_\infty)/\partial \log X = -\alpha$ — a $+10\%$ increase in X cuts reducible loss by about $\alpha \times 10\%$.

What bends the line: undertraining / bottlenecks (another resource is limiting ⇒ flatter apparent slope) · **data repetition / low quality** (flattens the data exponent) · **distribution shift or recipe changes** (move the slope *and* the floor — always report the training recipe).

Same template, three resources

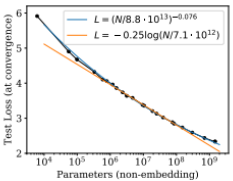


Figure 23 The trend for performance as a function of parameter count, $L(N)$, is fit better by a power law than by other functions such as a logarithm at a qualitative level.

Test loss vs. parameter count N — a clean power law.

Source: Kaplan et al. (2020), Fig. 23

Each resource obeys the *same* template — only the exponent changes:

Resource	Single-factor law
Params N	$L \approx L_\infty + A\,N^{-\alpha}$
Data D	$L \approx L_\infty + B\,D^{-\beta}$
Compute C	$L \approx L_\infty + E\,C^{-\gamma}$

Same *shape*, three *slopes* (α, β, γ). Measured exponents are small (~ 0.05–0.1): each $10\times$ of a resource buys a modest but *steady* drop in loss.

The aggregate law: combining the three axes

Add the three single-factor power laws you've just seen — under one fixed training *recipe* — into a single expression for the reducible loss:

$$L(N, D, C) \approx \underbrace{L_\infty}_{\text{shared floor}} + \underbrace{A\,N^{-\alpha}}_{\text{params}} + \underbrace{B\,D^{-\beta}}_{\text{data}} + \underbrace{E\,C^{-\gamma}}_{\text{compute}}$$

- Each term is one of the **single-factor laws** from the previous slides; there is **one shared** irreducible floor L_∞.
- Hold two resources abundant and you **recover a single law** — a straight line of slope $-\alpha$, $-\beta$, or $-\gamma$ on log–log.
- Together they let you **forecast** loss and **plan budgets**: with the coupling $C \approx kND$ (next) they pin down the **compute-optimal** $N^*(C)$ and $D^*(C)$.

The three axes are coupled: $C \approx kND$

Every token passes through *every* parameter.

- **One token.** Processing it uses all N weights, and each weight does the same tiny piece of arithmetic — a multiply and an add. So one token costs $k \cdot N$ FLOPs.
- **Now D tokens.** The model does not change between tokens, so that cost simply repeats:

$$C = \underbrace{kN}_{\text{per token}} \times D.$$

- **So:** compute is a *product*, not a sum.

k is just the exchange rate
FLOPs per parameter, per token. $k \approx 6$ when **training** (a forward pass, plus a backward pass costing twice as much); $k \approx 2$ at **inference**.

Consequence. Fix the budget C and the two axes are no longer free:

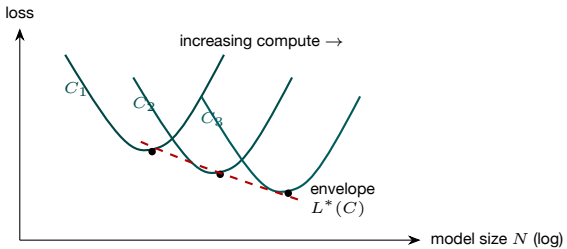
$$D = \frac{C}{kN}$$

A bigger model *must* read less data.

FLOP = **FL**oating-point **OP**eration — one multiply or one add. C counts the *total* arithmetic of the run, not FLOP/s (a *rate*).

Caveats: mixture-of-experts sends each token to only a few experts, so the *active* N sits below the stored N; attention adds an $O(L^2)$ term. Neither disturbs the $N \times D$ structure.

The compute-optimal frontier, in one picture



Why a U? Each curve fixes C, so $D = C/(kN)$. Put that into the aggregate law:

$$L(N) = L_\infty + \underbrace{A\,N^{-\alpha}}_{\downarrow \text{ in } N} + \underbrace{B\left(\frac{kN}{C}\right)^{\beta}}_{\uparrow \text{ in } N}$$

One term down, one up $\Rightarrow$ a single minimum. Sliding *right* buys parameters by **spending tokens**.

- The **minimum** of each curve is the compute-optimal $N^*(C)$; connect the minima $\Rightarrow$ the **envelope** $L^*(C)$, itself a power law.
- The optimum slides **down and to the right**: more compute $\Rightarrow$ bigger model *and* more data.

Compute-optimal allocation, derived

Set-up. Minimize reducible loss subject to the compute budget:

$$\min_{N,D} \; A\,N^{-\alpha} + B\,D^{-\beta} \quad \text{s.t.} \quad ND = \frac{C}{k}.$$

Solve (Lagrange / substitution) $\Rightarrow$ the optimal split:

$$N^*(C) \propto C^{\frac{\beta}{\alpha+\beta}}, \qquad D^*(C) \propto C^{\frac{\alpha}{\alpha+\beta}}.$$

Envelope. Substitute back: the best loss per FLOP is again a power law,

$$L^*(C) = L_\infty + K\,C^{-\eta}, \qquad \eta = \frac{\alpha\beta}{\alpha+\beta}.$$

Read it: if $\alpha \approx \beta$, then N^* and D^* both scale like $\sqrt{C}$ — **grow model and data in lockstep**.

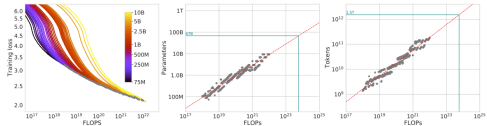


Figure 2 | **Training curve envelope.** On the **left** we show all of our different runs. We launched a range of model sizes going from 70M to 10B, each for four different cosine cycle lengths. From these curves, we extracted the envelope of minimal loss per FLOP, and we used these points to estimate the optimal model size (**center**) for a given compute budget and the optimal number of training tokens (**right**). In green, we show projections of optimal model size and training token count based on the number of FLOPs used to train *Gopher* (5.76×10^{23}).

Training-curve envelope. Each run traces loss vs. FLOPs; the lower *envelope* (the compute-optimal frontier) pins down $N^*(C)$ and $D^*(C)$.

Hoffmann et al. (2022), Training Compute-Optimal LLMs.

Kaplan vs. Chinchilla: same formula, opposite advice

Slide 47's rule needs two numbers, α and β. Two teams measured them — and the *same* formula then gave opposite instructions.

	optimal split	$10\times$ compute buys	tokens / param
Kaplan (2020)	$N^* \propto C^{0.73},\ D^* \propto C^{0.27}$	$5.4\times$ model, $1.9\times$ data	*falls* as $C^{-0.46}$
Chinchilla (2022)	$N^* \propto C^{0.50},\ D^* \propto C^{0.50}$	$3.2\times$ model, $3.2\times$ data	**fixed** ≈ 20

What the old rule cost

GPT-3: 175B parameters on ∼300B tokens $= 1.7$ tokens/param — roughly $12\times$ below Chinchilla's 20. A giant that never read enough.

What the new rule bought

Chinchilla (70B) beat Gopher (280B) at the *same* compute — $4\times$ smaller and better, purely by moving budget from parameters to tokens.

The last column *is* the disagreement: under Kaplan the token-to-parameter ratio shrinks as you scale, so bigger models should read relatively less. Under Chinchilla it never moves.

Why the flip — and what survives it

The formula never changed — only the *measured* α, β. The optimal split is a *ratio* of exponents, so a modest measurement bias swings the allocation a long way.

What Hoffmann et al. (2022) did differently:

- **Matched the schedule to the run.** Kaplan used one cosine learning-rate schedule for every run; a cycle much *longer* than the run **overstates** its final loss. Short runs were scored wrongly — and a wrong loss curve gives a wrong exponent.
- **Counted every parameter.** Kaplan's N excluded embeddings — a large share of a *small* model, distorting the low-N end of the curve.
- **Measured far more, three ways.** $\sim$400 runs, analysed by fixed-size sweeps, IsoFLOP profiles *and* a parametric fit — all landing on $\approx 0.5/0.5$.

Two lessons that outlast the numbers:

- An exponent measures **one training recipe**, not a constant of nature — pin down optimizer, schedule and data before quoting one.
- Empirical science **self-corrects**: one better measurement redirected billions of dollars of compute — and a 2024 replication has since challenged Chinchilla's own fit.

Practical playbook — and a prediction you can make

Playbook (rules of thumb):

- **Compute-optimal (Chinchilla):** scale N and D together, ≈ 20 tokens/param; doubling $C \Rightarrow \sim 1.4\times$ bigger model on $\sim 1.4\times$ more data.
- **Inference-optimal:** if serving dominates cost, deliberately **over-train a smaller model** (more D, less N) — training is one-time, inference scales with usage.
- **Sample efficiency:** bigger models hit a target loss in *fewer* tokens — larger is more data-efficient *per token*.

Exercise. A pilot gives reducible loss $L - L_\infty = 0.40$ at $N = 10^8$ and 0.34 at $N = 10^9$. Estimate α, then predict the loss at $N = 10^{10}$. *(Slope on log–log:* $\alpha = \log(0.40/0.34)/\log 10 \approx 0.07$*; extrapolate one more decade.)*

What Scaling Laws Really Tell Us?

Advantages:

- They provide **empirical regularities**: loss typically falls as a **power law** with model size and data.
- They guide **budget allocation**: the **compute-optimal tradeoff** between parameters, data and training time.
- They enable **predictability**: small pilot runs can forecast returns at larger scale.
- They highlight **diminishing returns** and when to spend on **data quality** instead of more parameters.

Limitations:

- They are **descriptive, not causal** explanations of why models work.
- Exponents **depend on task, data and architecture**, and shift under distribution shift.
- They say **nothing about optimization stability** (learning rates, curvature, mode collapse).
- They do **not guarantee generalization** or sample efficiency; they average over many confounders.
- **Emergence:** some capabilities appear abruptly at scale — a smooth loss curve can hide sharp task-level jumps (still debated).

So if scaling alone isn't enough, what explains generalization and efficient training? That is Part II: implicit bias of SGD, flat vs sharp minima, margins, regularization, and how architecture + data shape out-of-sample performance.

PART II

Generalization & Spectral Analysis

Double descent, implicit bias, the neural tangent kernel, and what weight spectra say about generalization.

Part II — Contents

Classical statistical learning theory

Double descent

Double descent in linear regression

Implicit bias of gradient descent

Linearizing neural networks: the NTK

Spectral theory of generalization

Roadmap

Classical statistical learning theory

Double descent

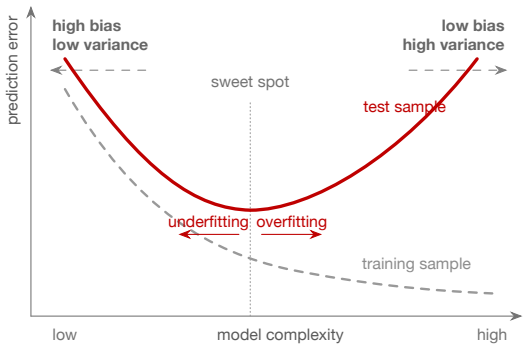
Double descent in linear regression

Implicit bias of gradient descent

Linearizing neural networks: the NTK

Spectral theory of generalization

The classical picture: one U-shaped curve



The textbook bias–variance trade-off (after Hastie et al., *ESL*).

- Training error falls monotonically with model complexity; **test error is U-shaped**.
- Left of the minimum: **underfitting** (high bias, low variance).
- Right of the minimum: **overfitting** (low bias, high variance).
- Classical prescription: pick capacity at the *sweet spot*; never let the model fit the training data exactly.

Deep networks routinely *interpolate* the training data — and still generalize. Part II asks why.

Classical generalization bounds

With probability at least $1-\delta$ over an i.i.d. sample of size N, for the learned hypothesis g (true risk $\mathfrak{R}$, empirical risk $\hat{\mathfrak{R}}$):

	Finite $\mathcal{H}$	**Infinite $\mathcal{H}$**
Realizable	$\mathfrak{R}(g) \;\le\; \frac{1}{N}\left[\log\lvert\mathcal{H}\rvert + \log\frac{1}{\delta}\right]$	$\mathfrak{R}(g) \;<\; \frac{VC(\mathcal{H})\log 2N + \log\frac{1}{\delta}}{N}$
Agnostic	$\mathfrak{R}(g) \;<\; \hat{\mathfrak{R}}(g) + \sqrt{\frac{1}{2N}\log\frac{2\lvert\mathcal{H}\rvert}{\delta}}$	$\mathfrak{R}(g) \;<\; \hat{\mathfrak{R}}(g) + \sqrt{\frac{VC(\mathcal{H})\log\frac{2N}{VC(\mathcal{H})} + \log\frac{4}{\delta}}{N}}$

- The complexity currency is $\log\lvert\mathcal{H}\rvert$ for finite classes and the **VC dimension** $VC(\mathcal{H})$ for infinite ones.
- Message: generalization gap $\sim \sqrt{\text{capacity}/N}$ — *more capacity requires more data*.

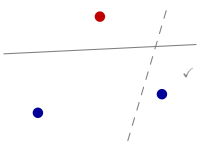
VC dimension

What is the Vapnik–Chervonenkis dimension $VC(\mathcal{H})$ of a hypothesis space $\mathcal{H}$?

- A **dichotomy** of a set S is a partition of S into two disjoint subsets.
- S is **shattered** by $\mathcal{H}$ if and only if for *every* dichotomy of S there exists some hypothesis in $\mathcal{H}$ consistent with it.
- $VC(\mathcal{H})$ is the size of the **largest finite subset of $\mathcal{X}$ shattered by $\mathcal{H}$**. If arbitrarily large finite sets can be shattered, $VC(\mathcal{H}) = \infty$.

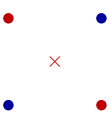
VC measures *worst-case shattering power*; higher VC implies a greater sample requirement or stronger regularization for good generalization.

half-planes in $\mathbb{R}^2$: 3 points shattered



every $\pm$ labeling separable $\Rightarrow VC \geq 3$

4 points: the XOR labeling



no half-plane realizes it $\Rightarrow VC = 3$

VC dimension: examples

Hypothesis class	VC dimension (approx/exact)
Half-line thresholds on $\mathbb{R}$, $h_a(x) = \mathbf{1}\{x \geq a\}$	1
Intervals on $\mathbb{R}$ (positive on $[a, b]$)	2
Union of k intervals on $\mathbb{R}$	$2k$
Affine hyperplanes in $\mathbb{R}^d$ (with bias)	$d + 1$
Axis-aligned rectangles in $\mathbb{R}^d$	$2d$ ($\Rightarrow$ in $\mathbb{R}^2$: 4)
Euclidean balls in $\mathbb{R}^d$	$d + 1$
Depth-L axis-aligned decision trees	$O(2^L \log 2^L)$ (grows with #leaves)
1-Nearest Neighbor in $\mathbb{R}^d$	∞
Degree-p polynomial separators in $\mathbb{R}^d$	$\binom{d+p}{p}$
ReLU (piecewise-linear) NN with W weights	$\Theta(W \log W)$ (tight up to constants)
Linear separators with margin γ on radius-R data	$\lesssim (R/\gamma)^2$ (dimension-free)

A ReLU net with millions of weights has **huge** VC dimension — the classical bounds above are *vacuous* for deep learning.

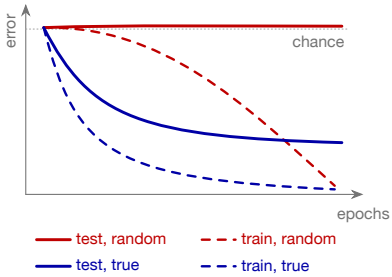
An experiment the theory cannot explain

The set-up. A standard image classifier and $50{,}000$ labelled photos.

Run 1 — scrambled labels. Shuffle every label at random: a cat photo is relabelled "truck". *There is no pattern left to learn.* Train anyway $\Rightarrow$ **zero training error**: it memorised $50{,}000$ arbitrary answers. On new photos it is no better than guessing.

Run 2 — the real labels. Same architecture, same optimizer, same number of steps. Now it **generalizes well** — with or without any explicit regularization.

Two completely different outcomes from the *same* model class and the *same* algorithm. Only the **data** changed.



Red = scrambled labels: training error falls to zero, test error never leaves chance. That gap *is* memorisation.

Schematic after Zhang et al. (2017). Network: Inception; data: CIFAR-10 — $50{,}000$ photos, 10 classes.

Zhang et al. (2017) — Understanding Deep Learning Requires Rethinking Generalization

Why that breaks the classical bound

Two slides ago the guarantee was

$$\text{test error} \;\leq\; \text{train error} + \sqrt{\text{capacity}/N}.$$

What "capacity" means. How many different labellings of the N training points the class *could* fit. Run 1 fitted a **random** labelling — so the class can fit essentially **every** labelling. Capacity is as large as it gets.

Consequence. The square-root term blows up, and the guarantee collapses to a statement no stronger than "your error is at most 100%". It is still *true*. It simply rules nothing out.

The sharper problem

The right-hand side depends only on the *class* $\mathcal{H}$ and the sample size N — never on the labels. Both runs used the same $\mathcal{H}$ and the same N.

$\Rightarrow$ the bound returns the **identical number** for Run 1 and Run 2, yet one memorises and the other generalizes. No bound of this shape can tell the two apart.

A uniform bound describes the **menu** — every function the class could have produced. It never asks **which** function the algorithm actually picked, or what the **data** looked like. Generalization must depend on both.

Roadmap

Classical statistical learning theory

Double descent

Double descent in linear regression

Implicit bias of gradient descent

Linearizing neural networks: the NTK

Spectral theory of generalization

Double descent: the modern risk curve

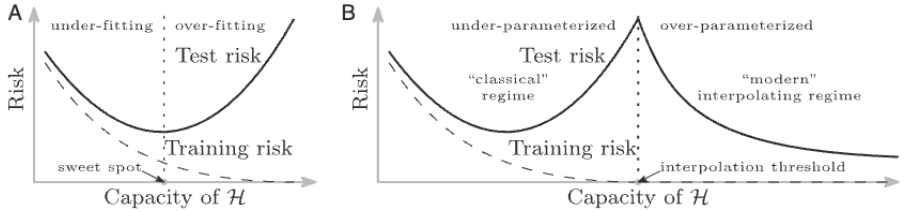


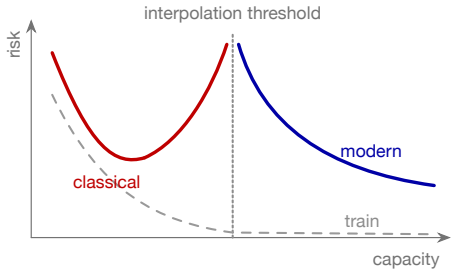
Fig. 1. Curves for training risk (dashed line) and test risk (solid line). (*A*) The classical U-shaped risk curve arising from the bias–variance trade-off. (*B*) The double-descent risk curve, which incorporates the U-shaped risk curve (i.e., the "classical" regime) together with the observed behavior from using high-capacity function classes (i.e., the "modern" interpolating regime), separated by the interpolation threshold. The predictors to the right of the interpolation threshold have zero training risk.

- **(A)** the classical U-shaped test-risk curve from the bias–variance trade-off;
- **(B)** the **double-descent** curve: the classical regime, the *interpolation threshold* where the model first fits the training data exactly, and a *second descent* in the modern over-parameterized regime.

Belkin, Hsu, Ma, Mandal (PNAS 2019)

Reading the double-descent curve

- As capacity grows from small to moderate, test error first **decreases, then increases** — the classical U.
- There is a point where the model **exactly fits** the training data while test error is near its *highest*.
- Beyond this point, further increasing capacity makes test error **decrease again**.
- Training error keeps decreasing with capacity and remains ≈ 0 after the exact fit.
- The behavior is therefore not a single U-shape but a **"double descent"** in test error as capacity grows.
- Models *far larger* than needed to interpolate can achieve **lower** test error than smaller models.



Keep this picture in mind — the rest of the lecture explains each segment of it.

Random Fourier features: the construction

1. Draw N frequencies once, at random.

$$w_1, \dots, w_N \overset{\text{iid}}{\sim} \mathcal{N}\big(0, \sigma^{-2} I_d\big), \qquad b_j \sim U[0, 2\pi]$$

Never trained — sampled once, then frozen.

2. Push every input through them.

$$\phi(x) = \big(\cos(w_1^\top x + b_1), \dots, \cos(w_N^\top x + b_N)\big) \in \mathbb{R}^N$$

3. Fit a linear model *on top* — only a is learned.

$$\hat{f}(x) = \sum_{j=1}^{N} a_j \, \phi_j(x) = a^\top \phi(x), \qquad \min_a \|\Phi a - y\|^2$$

Nonlinear in x, **linear in** a — least squares still applies.

What is augmented. The design matrix changes shape, nothing else:

$$\underbrace{X \in \mathbb{R}^{n \times d}}_{d=784 \text{ pixels}} \longrightarrow \underbrace{\Phi \in \mathbb{R}^{n \times N}}_{N \text{ up to } 10^5}$$

Same data, same loss, same estimator — only N moves: a capacity dial *independent of* n.

Why "Fourier". Bochner: a shift-invariant kernel is the Fourier transform of a density $p(w)$, so random frequencies reproduce it:

$$\tfrac{1}{N} \textstyle\sum_j \phi_j(x)\phi_j(x') \xrightarrow{N \to \infty} k(x, x')$$

With $w \sim \mathcal{N}(0, \sigma^{-2} I)$ that kernel is the Gaussian RBF — the features are a **Monte-Carlo sketch of a kernel machine**.

Three regimes in one dial: $N < n$ least squares · $N = n$ the spike · $N > n$ **minimum-norm** interpolation.

Rahimi & Recht (2007), Random Features for Large-Scale Kernel Machines

Evidence (1): random Fourier features on MNIST

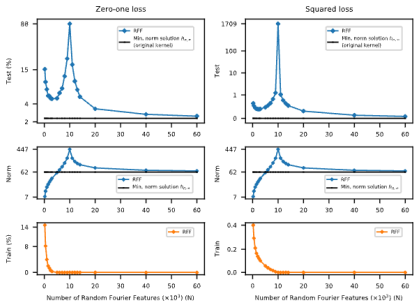


Fig. 2. Double-descent risk curve for the RFF model on MNIST. Shown are test risks (log scale), coefficient ℓ_2 norms (log scale), and training risks of the RFF model predictors $\hat{h}_{n,N}$ learned on a subset of MNIST ($n = 10^4$, 10 classes). The interpolation threshold is achieved at $N = 10^4$.

- A **linear model in random features**: increasing the number of features N moves from under- to over-parameterized.
- Test risk **peaks at the interpolation threshold** ($N = n = 10^4$) and then descends again — under both zero-one and squared loss.
- The coefficient norm peaks there too, then falls toward the **minimum-norm** kernel solution.

Belkin et al. (PNAS 2019)

Evidence (2): neural networks and random forests

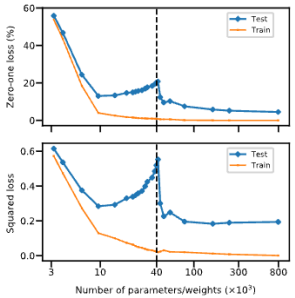


Fig. 3. Double-descent risk curve for a fully connected neural network on MNIST. Shown are training and test risks of a network with a single layer of H hidden units, learned on a subset of MNIST ($n = 4 \cdot 10^3$, $d = 784$, $K = 10$ classes). The number of parameters is $(d+1) \cdot H + (H+1) \cdot K$. The interpolation threshold (black dashed line) is observed at $n \cdot K$.

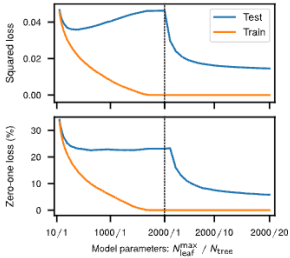


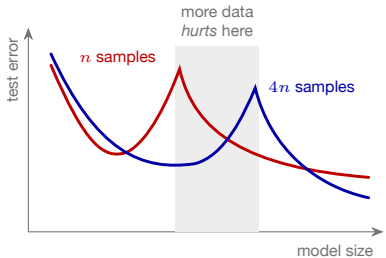
Fig. 4. Double-descent risk curve for random forests on MNIST. The double-descent risk curve is observed for random forests with increasing model complexity trained on a subset of MNIST ($n = 10^4$, 10 classes). Its complexity is controlled by the number of trees N_{tree} and the maximum number of leaves allowed for each tree $N_{\text{leaf}}^{\max}$.

- **Neural network:** the same peak at interpolation ($n \cdot K$ parameters), then a second descent.
- **Random forest:** more trees / leaves shows the same peak-then-decline pattern.
- Knob differs by family; the **pattern is consistent**.

Belkin et al. (PNAS 2019)

Evidence (3): modern deep networks

- **Model-wise:** ResNets and Transformers (CIFAR, IWSLT) show the same peak as width grows — especially visible with label noise.
- **Epoch-wise:** the *same* network traverses a double descent over *training time* — training longer can first hurt, then help.
- **Sample-wise:** more data moves the interpolation threshold, so near the peak **more data can hurt**.
- Unified by *effective model complexity*: the peak sits wherever training first barely fits the data.



Schematic after Nakkiran et al. (2019).

Double descent is **generic** — across model size, training time, and sample size.

Nakkiran, Kaplun, Bansal, Yang, Barak, Sutskever (2019) — Deep Double Descent

Roadmap

Classical statistical learning theory

Double descent

Double descent in linear regression

Implicit bias of gradient descent

Linearizing neural networks: the NTK

Spectral theory of generalization

A tractable model: least squares with weak features

Regression with response y linear in D real variables $\boldsymbol{x} = (x_1, \ldots, x_D)$ plus noise:

$$y \;=\; \boldsymbol{x}^\top \boldsymbol{\beta} + \sigma\varepsilon \;=\; \textstyle\sum_{j=1}^{D} x_j \beta_j + \sigma\varepsilon, \qquad \boldsymbol{\beta} \in \mathbb{R}^D.$$

- Given n i.i.d. copies $\{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$, fit a linear model using only a **subset** $T \subseteq [D]$ of $p = |T|$ variables — $\boldsymbol{X}_T$ the $n \times p$ restricted design — via the **Moore–Penrose pseudoinverse**:

$$\hat{\boldsymbol{\beta}}_T \;:=\; \boldsymbol{X}_T^{+}\, \boldsymbol{y}, \qquad \hat{\boldsymbol{\beta}}_{T^c} \;:=\; \boldsymbol{0}.$$

Moore–Penrose $\boldsymbol{X}^+$: among all $\boldsymbol{\beta}$ minimising $\|\boldsymbol{X}\boldsymbol{\beta} - \boldsymbol{y}\|_2$, it returns the one of smallest $\|\boldsymbol{\beta}\|_2$. $p \le n$ (full column rank): $\boldsymbol{X}^+ = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top$, ordinary least squares. $p > n$ (full row rank): $\boldsymbol{X}^+ = \boldsymbol{X}^\top (\boldsymbol{X}\boldsymbol{X}^\top)^{-1}$, which interpolates.

- So for $p > n$ it is the **minimum-norm** interpolant — exactly the solution gradient descent finds (next section).
- Sweep p from 1 to D: the model moves from under- to over-parameterized *within ordinary linear regression*.

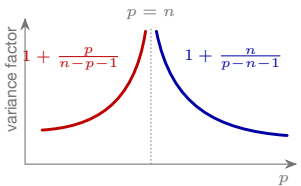
Belkin, Hsu, Xu (2020) — Two models of double descent for weak features

The risk can be computed exactly

Theorem (Belkin–Hsu–Xu). Let $\boldsymbol{x}$ be standard normal in $\mathbb{R}^D$, ε standard normal independent of $\boldsymbol{x}$, and $y = \boldsymbol{x}^\top \boldsymbol{\beta} + \sigma\varepsilon$. Pick any $p \in \{0, \dots, D\}$ and $T \subseteq [D]$ with $|T| = p$. The risk of $\hat{\boldsymbol{\beta}}$ is

$$\mathbb{E}\big[(y - \boldsymbol{x}^\top \hat{\boldsymbol{\beta}})^2\big] = \begin{cases} \big(\|\boldsymbol{\beta}_{T^c}\|^2 + \sigma^2\big)\Big(1 + \dfrac{p}{n-p-1}\Big) & \text{if } p \le n-2; \\ +\infty & \text{if } n-1 \le p \le n+1; \\ \|\boldsymbol{\beta}_T\|^2\Big(1 - \dfrac{n}{p}\Big) + \big(\|\boldsymbol{\beta}_{T^c}\|^2 + \sigma^2\big)\Big(1 + \dfrac{n}{p-n-1}\Big) & \text{if } p \ge n+2. \end{cases}$$

- $p \le n-2$: classical regime — variance factor $\frac{p}{n-p-1}$ grows as $p \uparrow n$.
- $p \approx n$: the design matrix is nearly singular — the risk **diverges** at the threshold.
- $p \ge n+2$: the factor $\frac{n}{p-n-1}$ *shrinks* as p grows — the **second descent**.



Random vs. prescient feature selection

Setup: $n = 40$, $D = 100$; least-squares regression on p features.

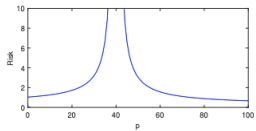


Figure 1: Plot of risk $\mathbb{E}[(y - \boldsymbol{x}^\star\hat{\boldsymbol{\beta}})^2]$ as a function of p, under the random selection model of T. Here, $\|\boldsymbol{\beta}\|^2 = 1$, $\sigma^2 = 1/25$, $D = 100$, and $n = 40$.

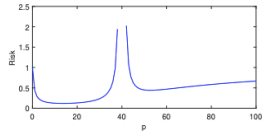


Figure 2: Plot of risk $\mathbb{E}[(y - \boldsymbol{x}^\star\hat{\boldsymbol{\beta}})^2]$ as a function of p, under the "prescient" selection model of T. Here, $\|\boldsymbol{\beta}\|^2 = 1$, $\beta_j^2 \propto 1/j^2$, $\sigma^2 = 1/25$, $D = 100$, and $n = 40$.

Random feature choice. As p increases up to n, the model overfits and risk worsens — classical ascent without the initial dip. For $p > n$ the risk **decreases again**: the model perfectly fits the data while becoming *less* volatile.

Prescient choice (strongest features first). A "sweet spot" at low p, resembling the traditional descent, with higher risk in the interpolation regime. If features can be cherry-picked, few variables suffice; interpolation helps mostly when feature choice is *unbiased*.

Belkin, Hsu, Xu (2020); exposition after C. Sanford (2021)

Roadmap

Classical statistical learning theory

Double descent

Double descent in linear regression

Implicit bias of gradient descent

Linearizing neural networks: the NTK

Spectral theory of generalization

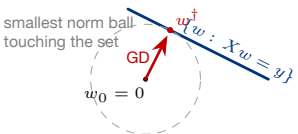
GD selects the minimum-norm interpolant

With $d > n$ (full row rank) infinitely many w satisfy $Xw = y$. One is distinguished — *three equivalent names for it*:

$$\underbrace{\arg\min_{w:\, Xw=y} \|w\|_2}_{\text{smallest interpolant}} \;=\; \underbrace{X^{+}y}_{\text{algebraic}} \;=\; \underbrace{\lim_{\lambda \downarrow 0} \arg\min_{w} \; \|Xw - y\|_2^2 + \lambda \|w\|_2^2}_{\text{ridgeless limit}} \;=:\; w^{\dagger}.$$

Proposition (implicit bias of GD). Run gradient descent on $L(w) = \frac{1}{2}\|Xw - y\|_2^2$ from $\boldsymbol{w_0 = 0}$ with $0 < \eta < 2/\lambda_{\max}(X^\top X)$. Then $w_t \to w^{\dagger}$.

Proof. $\nabla L(w) = X^\top(Xw - y) \in \mathrm{row}(X)$, so from $w_0 = 0$ every iterate stays in $\mathrm{row}(X)$. GD converges to an interpolant, and the only one in $\mathrm{row}(X)$ is $w^{\dagger}$. □



X^{+}**:** with $d > n$ and full row rank, $X^{+} = X^\top(XX^\top)^{-1}$. ($K^{+}$ likewise.)

RKHS analogue:

$f^{\dagger} = \arg\min_{f:\, f(X)=y} \|f\|_{\mathcal{H}} \Rightarrow f^{\dagger}(x) = k(x, X)^\top K^{+} y.$

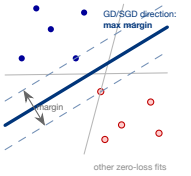
Bartlett, Long, Lugosi, Tsigler (2020) — Benign Overfitting in Linear Regression

Classification: implicit max-margin bias

Every separator of **separable** data reaches zero loss — one is distinguished, the **max-margin** (hard-SVM) one:

$$\hat{w} \;=\; \arg\min_{w} \|w\|_2 \quad \text{s.t.} \quad y_i\, x_i^\top w \;\geq\; 1 \;\; \forall i.$$

Itself a min-norm problem — the previous slide's principle, with "interpolates" → "classifies with margin".



Proposition (Soudry et al. 2018). GD on cross-entropy (any *exponential-tail* loss) over separable data: $\|w_t\| \to \infty$ — **no limit point** — yet the *direction* converges,

$$\frac{w_t}{\|w_t\|} \;\longrightarrow\; \frac{\hat{w}}{\|\hat{w}\|}, \qquad w_t = \hat{w}\log t + O(1).$$

Why the tail matters: well-classified points contribute exponentially little — the gradient is dominated by the support vectors.

Also holds for **SGD**. The $O(1/\log t)$ rate is why training **past zero error** keeps growing the margin.

Soudry et al. (2018) Nacson et al. (2019) — SGD on separable data

Roadmap

Classical statistical learning theory

Double descent

Double descent in linear regression

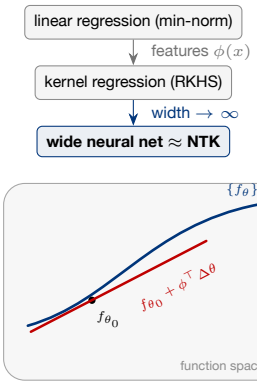
Implicit bias of gradient descent

Linearizing neural networks: the NTK

Spectral theory of generalization

Bridging to the Neural Tangent Kernel

- We now have the full story for **linear and kernel models**: they interpolate, exhibit double descent, and GD *selects* the min-norm / max-margin solution.
- **Natural question:** can we approximate deep nets as linear or kernel machines, so all of these insights *port over* to modern networks?
- **Idea:** look at *very wide* networks trained with *small steps*; training can be linearized around initialization and viewed through a kernel **induced by the network itself**.
- This leads to the **Neural Tangent Kernel (NTK)**: wide nets behave like kernel regressors with a network-defined kernel.



Wider network $\Rightarrow$ training stays closer to the tangent model ("lazy" regime).

Linearization around initialization

Setup (scalar output): data $X = \{x_i\}_{i=1}^n$, labels $y_i \in \mathbb{R}$; network $f_\theta : \mathbb{R}^d \to \mathbb{R}$ with $\theta \in \mathbb{R}^p$; initialization θ_0; change $\Delta\theta := \theta - \theta_0$.

Fixed "tangent" features: $\phi(x) := \nabla_\theta f_{\theta_0}(x) \in \mathbb{R}^p$ — the gradient of the output w.r.t. the parameters, evaluated *once* at θ_0. **First-order approximation around θ_0:**

$$f_\theta(x) \approx \underbrace{f_{\theta_0}(x)}_{\substack{\text{constant} \\ \text{frozen at init}}} + \underbrace{\phi(x)^\top}_{\substack{\text{fixed features} \\ \text{frozen at init}}} \underbrace{\boldsymbol{\Delta\theta}}_{\substack{\textbf{the only} \\ \textbf{unknown}}} .$$

Everything in grey is computed *once* at θ_0 and never moves; only $\Delta\theta$ is learned. So the right-hand side is **ordinary linear regression in the features** $\phi(x)$ — exactly the object of the last ten slides, with an offset.

Compare the RFF slide: there ϕ came from *random* frequencies; here $\phi = \nabla_\theta f_{\theta_0}$ is handed to us by the network's own initialization. **Wide** + **small steps** is what keeps ϕ from moving (the "lazy" regime) — and that is the whole bridge: NN $\approx$ NTK kernel regression, so the double-descent and benign-overfitting results largely carry over.

Jacot, Gabriel, Hongler (NeurIPS 2018) — Neural Tangent Kernel

NTK: training dynamics and closed-form limit

Neural Tangent Kernel: $K(x, x') := \phi(x)^\top \phi(x') \in \mathbb{R}$; Gram matrix $K(X, X) \in \mathbb{R}^{n \times n}$, $K_{ij} = K(x_i, x_j)$; cross-kernel $K(x, X) \in \mathbb{R}^n$.

Training dynamics (squared loss, gradient flow). Let $f_t(X) \in \mathbb{R}^n$ collect the predictions. In the lazy regime,

$$\frac{df_t(X)}{dt} = -K(X, X)\big(f_t(X) - y\big) \qquad \text{— kernel gradient descent with kernel } K.$$

Closed-form limit (interpolation, $K(X, X)$ invertible):

$$f_\infty(x) = K(x, X)\, K(X, X)^{-1} y, \qquad f_\lambda(x) = K(x, X)\big(K(X, X) + \lambda I\big)^{-1} y, \quad \lambda > 0.$$

- The trained wide network behaves like **kernel regression with the NTK**.
- Everything we established — min-norm interpolation, double descent, implicit bias — now speaks to *neural networks*.
- **Caveat:** this is the *lazy* regime — finite-width nets also *learn features*, where they beat fixed kernels.

Roadmap

Classical statistical learning theory

Double descent

Double descent in linear regression

Implicit bias of gradient descent

Linearizing neural networks: the NTK

Spectral theory of generalization

Not all interpolation is benign: a taxonomy

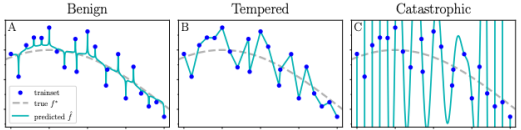
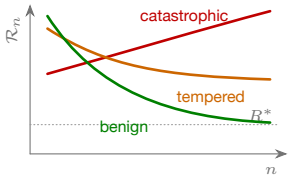


Figure 1: **As $n \to \infty$, interpolating methods can exhibit three types of overfitting. (A)** In *benign overfitting*, the predictor asymptotically approaches the ground-truth, Bayes-optimal function. Nadaraya-Watson kernel smoothing with a singular kernel, shown here, is asymptotically benign. **(B)** In *tempered overfitting*, the regime studied in this work, the predictor approaches a constant test risk greater than the Bayes-optimal risk. Piecewise-linear interpolation is asymptotically tempered. **(C)** In *catastrophic overfitting*, the predictor generalizes arbitrarily poorly. Rank-n polynomial interpolation is asymptotically catastrophic.

$\mathcal{R}_\infty := \lim_{n\to\infty} \mathcal{R}_n$ (risk of the interpolating rule as data grows):

	Regression	Classification
Benign	$\mathcal{R}_\infty = R^*$	$\mathcal{R}_\infty = R^*$
Tempered	$\mathcal{R}_\infty \in (R^*, \infty)$	$\mathcal{R}_\infty \in \left(R^*, 1 - \frac{1}{K}\right)$
Catastrophic	$\mathcal{R}_\infty = \infty$	$\mathcal{R}_\infty = 1 - \frac{1}{K}$



Mallinar et al. (2024) — Benign, Tempered, or Catastrophic: A Taxonomy of Overfitting

Which methods land where

Benign	Tempered	Catastrophic
Early-stopped DNNs	**Interpolating DNNs**	**Gaussian kernel regression**
Kernel regression with ridge	**Laplace kernel regression**	Critically-parameterized regression
k-NN ($k \sim \log n$)	**ReLU NTKs**	
Nadaraya–Watson smoothing (Hilbert kernel)	k-NN (constant k), simplicial interpolation	

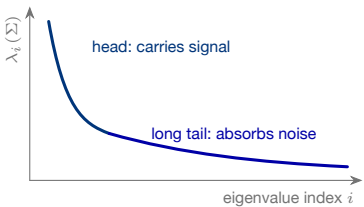
- Interpolating deep nets are typically **tempered**: not optimal, but far from catastrophic — overfitting is *mild*, not fatal.
- Which regime you get depends on the **spectrum** of the kernel / model — motivating a spectral look at trained networks.

Mallinar et al. (2024)

What decides the regime? Spectra

Benign overfitting has a spectral characterization (linear / kernel case): for min-norm interpolation, benignness is governed by the *effective ranks* of the data covariance spectrum (Bartlett et al. 2020):

- a few **large eigenvalues** must carry the signal directions, and
- a **long tail of many small eigenvalues** must exist to spread the noise thinly across unimportant directions — *self-induced regularization*.
- Too short a tail, or no signal head $\rightarrow$ tempered / catastrophic.



So far, the spectrum of the **data / kernel** decides generalization — a theorem. Next — *semi-empirical*: can we read a trained network's health off its **weight-matrix spectra**?

Bartlett, Long, Lugosi, Tsigler (2020) — Benign Overfitting in Linear Regression

Why heavy-tailed weight spectra signal generalization

Claim (intuition). Trained nets that generalize often have **heavy-tailed layer spectra**: for a layer matrix W, the eigenvalue density of $W^\top W$ has a power-law tail $\rho(\lambda) \propto \lambda^{-\alpha}$. This scale-free signature suggests the layer learned *multi-scale structure* rather than memorizing noise.

What we measure

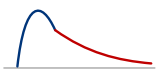
- For each weight matrix W: eigenvalues of $W^\top W$ (the empirical spectral density, ESD); fit a power law to its **tail**; record the tail index $\hat{\alpha}$ and its per-layer trends.

Why this makes sense (high-level mechanisms)

- **Scale-free structure in data:** natural data (images, language, time series) have long-range, multi-scale correlations $\rightarrow$ many meaningful directions with decaying strength $\rightarrow$ power-law spectra.
- **Implicit self-regularization:** stochastic optimization distributes energy across modes (near "criticality"), avoiding over-concentration in a few brittle directions.
- **Robust representation:** heavy tails imply high *effective rank* with decay — the model uses many features a little, rather than a few features too much.

Martin & Mahoney (2021) — Implicit Self-Regularization in Deep Neural Networks

If the spectrum is *not* heavy-tailed?

ESD	Spectrum look	Typical situation	What it suggests
	**MP-like bulk** (light-tailed, near random)	Under-trained / excessive regularization / poor optimization	No rich structure yet → *underfit*
	**Spiky** (few huge outliers on a flat bulk)	Shortcut features, leakage, correlation traps, memorization	Brittle directions dominate → *overfit / poor robustness*
	**Healthy heavy tail** (smooth decay)	Well-trained on structured data	Broad, multi-scale features → *better generalization*

MP = Marchenko–Pastur: the spectrum of a purely random matrix — the "no learned structure" baseline from random matrix theory.

SETOL & WeightWatcher: what it says, how to use it

Key result (informal). SETOL derives a **layer-quality metric** via a one-step Exact Renormalization Group (ERG) construction and shows it **tracks** WeightWatcher's per-layer tail-index $\hat{\alpha}$.

Regimes — what α is telling you

- **Ideal learning:** healthy heavy-tailed range, $\alpha \approx 2$ ($\sim$2–6 acceptable).
- **Overfit / memorization:** extremely heavy tails $\rightarrow$ very small α (e.g., $\alpha < 2$).
- **Under-trained / noisy / correlation traps:** very large α or *spiky* ESDs (few huge outliers on a flat bulk).

How to use it (checklist)

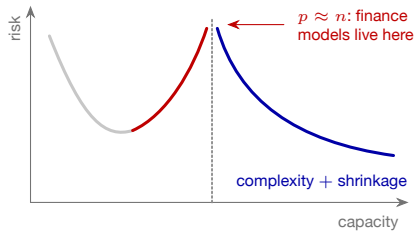
1. **Run WeightWatcher on checkpoints**: per-layer $\hat{\alpha}$, histograms, summaries.
2. **Read the dials:** red flags $\alpha < 2$ (memorizing) or $\alpha \gg 6$ (under-trained); inspect spikes/outliers.
3. **Decide actions:** data curation, LR/weight decay, depth/width, stopping time; re-check $\hat{\alpha}$ trends.

Theory (SETOL) and practice ($\hat{\alpha}$ from ESD fits) agree
— a fast, *data-free* QA **without touching the eval set**.

Martin & Hinrichs — SETOL WeightWatcher (GitHub)

What this means for quantitative finance

- Finance data: **small n, low signal-to-noise, non-stationary**. Kitchen-sink regressions and factor zoos often operate *near* $p \approx n$ — the **worst** point on the curve.
- **Shrinkage is not optional**: ridge / early stopping move an interpolating fit from tempered or catastrophic toward benign.
- **Overparameterization is not the enemy**: with proper shrinkage, rich models can dominate sparse ones — the "virtue of complexity" in return prediction.
- **Data-free QA**: weight-spectra diagnostics track model health *without touching the eval set* — backtest reads spend statistical budget.



Kelly, Malamud, Zhou (2024) — The Virtue of Complexity

PART III

The Virtue of Complexity

Does the Part II story survive contact with returns? Kelly, Malamud & Zhou (2024) say yes — and put a Sharpe ratio on it.

Roadmap

The claim

What is actually traded

The evidence

The lesson

The thesis in two sentences

- Once you admit that the true pricing signal is complex, **“Occam’s razor” is misleading**.
- Rich, high-dimensional models **with shrinkage** are not a vice; they are a *virtue* — they systematically deliver better trading performance than simple, elegant regressions.

Two worlds — and everything turns on which one you are in

Correctly specified

Your signal set *contains* the truth.

- The signals you regress on are exactly the ones generating the true expected return.
- With infinite data the regression **recovers the truth** exactly.
- So every error is **finite-sample estimation noise** — nothing is missing from the model.
- $\Rightarrow$ complexity is a **pure statistical cost**: classic double descent, OOS R^2 plunging near $c \approx 1$, and you never beat the oracle.

Misspecified

Your signal set *cannot* represent the truth.

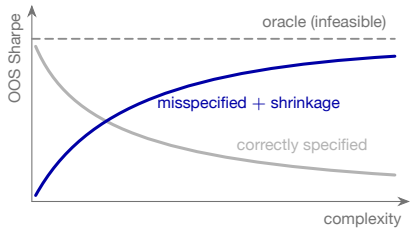
- Returns depend on a richer, nonlinear function of signals than the ones you use.
- Even with infinite data a gap remains: **approximation error**, *on top of* estimation noise.
- Adding signals **removes approximation error** — a source of gain the correctly specified world does not have.
- $\Rightarrow$ with shrinkage that gain **outruns** the added noise: OOS R^2 and Sharpe **increasing and concave**.

Same models, same estimator, *opposite* verdicts on complexity — decided entirely by whether your signals contain the truth. **No return model is correctly specified**, so the right-hand column describes return prediction. Everything that follows tests it.

The same curve, with money on the axis

Part II's phenomenon was about *test risk*. The finance version: does complexity pay in **out-of-sample Sharpe**?

- **If the model were correctly specified** — the true conditional mean really is linear in your signals — complexity is a *pure statistical cost*. You get the textbook double descent, and never beat the oracle.
- **But no return model is correctly specified.** The true DGP is unknown and nonlinear, so every small linear model carries **misspecification error**.
- Add signals and that error falls. With **shrinkage** the fall *outruns* the added noise — so OOS R^2 and Sharpe are **increasing and concave** in complexity.



Not a claim that overfitting is harmless — a claim that once you admit the model is wrong, the bias you remove is worth the variance you add.

Roadmap

The claim

What is actually traded

The evidence

The lesson

The market-timing strategy

Each period, the strategy chooses how much to invest in a **single risky asset** (e.g. the market index) based on the signals observed at that time.

1. Forecast

$\hat{\mu}_t = S_t' \hat{\beta}$

$\hat{\beta}$ estimated from past data.

2. Position

$\pi_t = S_t' \hat{\beta} = \hat{\mu}_t$

Weight (leverage) in the risky asset; the rest is in the risk-free asset.

3. Strategy return

$R^{\pi}_{t+1} = \pi_t \, R_{t+1}$

R_{t+1}: excess return of the risky asset.

Why call it "timing"?

- A **static** investor holds a fixed position (e.g. always 60% in the market).
- This strategy **changes its exposure over time**: a **large long** position when $\hat{\mu}_t \gg 0$; small, flat or short ($\pi_t \approx 0$ or $\pi_t < 0$) when the forecast is low or negative.
- It dynamically scales exposure to the *same* asset — market timing, not stock selection or static allocation.

Random Fourier features: from 15 predictors to any P

Base predictors. $G_t \in \mathbb{R}^{15}$: the Welch–Goyal predictors at time t.

Idea. Use RFFs to build a *much higher-dimensional* S_t from G_t — move smoothly from low- to high-dimensional models with the same economic inputs.

Construction (per random feature). Draw $\omega_i \sim N(0, I_{15})$, form $\omega_i' G_t$, and apply sine and cosine with bandwidth γ:

$$S_{i,t} = \left[\sin(\gamma\, \omega_i' G_t),\ \cos(\gamma\, \omega_i' G_t)\right].$$

Each ω_i gives **two** features.

Controlling complexity.

- 1 draw $\Rightarrow P = 2$: very low-dimensional.
- 5,000 draws $\Rightarrow P = 10{,}000$: very high-dimensional.
- Any P by choosing the number of ω_i's.

Interpretation.

- RFF approximates a general nonlinear $f(G_t)$ for $\mathbb{E}[R_{t+1} \mid G_t]$.
- Equivalent to a **two-layer neural network**: fixed random first layer (ω_i, sin/cos), learned linear second layer (β).
- The bridge between 15 economic predictors and the large-P linear models of the theory.

Roadmap

The claim

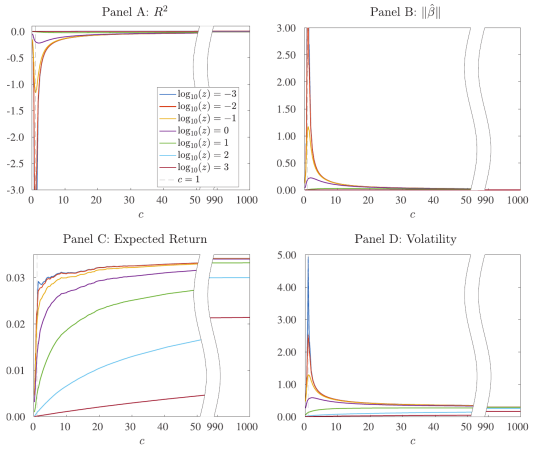
What is actually traded

The evidence

The lesson

OOS R^2 and $\|\hat{\beta}\|^2$ ($T = 12$)

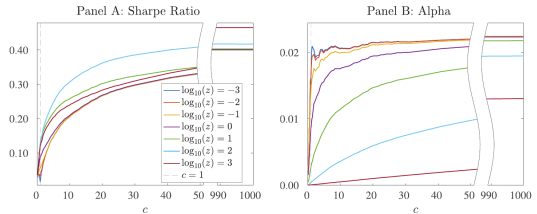
- Out-of-sample R^2 and coefficient norm vs. complexity c.
- R^2: **crashes** near the interpolation boundary $c \approx 1$; recovers and turns positive in the high-complexity regime, especially with ridge.
- $\|\hat{\beta}\|^2$: spikes at $c \approx 1$, declines for $c > 1$.
- **Matches the misspecified column** of the two-worlds slide: double descent in R^2, peak-then-shrink in $\|\hat{\beta}\|$, recovery at high c.



KMZ Fig. 7 — OOS market timing, $T = 12$, $P \in [2, 12{,}000]$, $\gamma = 2$.

Expected return, volatility, Sharpe, alpha & IR ($T = 12$)

- **Expected return** rises with c; nearly flat beyond $c = 1$ when almost ridgeless, keeps rising into very high c with stronger ridge.
- **Volatility** spikes near $c = 1$, then falls.
- **Sharpe & IR** rise with complexity (small dip at $c = 1$ for very low ridge), reaching $SR \gtrsim 0.4$, $IR \approx 0.3$.
- **Match to theory: yes** — richer models + shrinkage improve economic performance; Sharpe/IR increasing and concave.



KMZ Fig. 8 — alphas vs. a static position in the volatility-standardized market, $T = 12$.

Goyal & Welch (2008) vs. the virtue of complexity

Aspect	Goyal & Welch (2008) “kitchen sink”	Kelly–Malamud–Zhou (revisit of GW data)
Information set	15 monthly GW predictors	Same 15 predictors
Model class	Linear OLS (“kitchen-sink” regression)	Linear **ridge** + high-dimensional **RFF** (nonlinear)
Complexity regime	$P = 15$; short windows already near $c \approx 1$	Views GW as **near interpolation**; explores very high c with shrinkage
Metric emphasized	Out-of-sample **prediction** R^2	Out-of-sample **timing performance** (Sharpe, alpha, IR)
Baseline ($T = 12$)	OOS R^2 large **negative**; timing $SR \approx -0.1 \Rightarrow$ “return prediction fails”	Confirms bad R^2, but reads it as an **interpolation pathology**, not a lack of signal
Ridge on the kitchen sink	Not emphasized	With strong ridge ($z = 10^3$): R^2 slightly negative, but timing $SR \approx 0.46$, $IR \approx 0.33$, $t \approx 3.1$ vs. market
High-complexity nonlinear model	Not considered	RFF with $c \approx 1000$, $z = 10^3$: OOS $R^2 \approx 1\%$/month, $SR \approx 0.46$, $IR \approx 0.31$, $t \approx 2.5$ vs. best linear model
Overall message	OOS R^2 persistently negative $\Rightarrow$ timing with these predictors isn’t useful	The same data **do** support profitable timing once you (i) control interpolation with shrinkage and (ii) use the predictors in a high-dimensional nonlinear way

Key empirical numbers at high complexity

Quantity	Order of magnitude / takeaway
Max OOS Sharpe (high c, tuned ridge)	$SR \gtrsim 0.4$
Information ratio (alpha vs. buy-and-hold)	$IR \approx 0.3$ ($T = 12$); $IR \approx 0.25$ ($T = 60, 120$)
Alpha t-statistics	roughly 2.6–2.9 ($T = 12$); > 2.0 ($T = 60, 120$)
Near interpolation $c \approx 1$	$\|\hat{\beta}\|$, R^2 unstable; volatility spikes; SR dips
Large c with shrinkage	R^2 mostly positive; low volatility; SR, alpha, IR **strong**
Shape of empirical VoC curves	very close to the misspecified-world prediction

Roadmap

The claim

What is actually traded

The evidence

The lesson

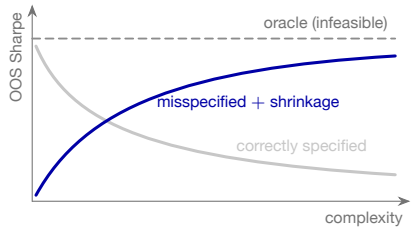
Implications: Occam's razor vs. Occam's blunder

Implications for practice

- *Not* a licence to add arbitrary junk predictors. Instead: include **all plausibly relevant signals**, and use **rich nonlinear models with shrinkage** rather than forcing simple linear specifications.
- Even a small raw predictor set gains substantially inside **highly parameterized nonlinear models**.

Occam's razor vs. Occam's blunder

- The traditional view (Box, 1976) warns against overparameterization and favours parsimony.
- This paper (and related ML theory): small models are preferable only if **truly correctly specified** — almost never the case.
- Under broad conditions **large models are preferable**; the lesson likely applies widely in finance and economics.



References — Parts I & II

Part I — Approximation & Scaling

- Cybenko (1989), Approximation by superpositions of a sigmoidal function, *Math. Control Signals Systems* 2(4):303–314.
- Telgarsky (2016), Benefits of depth in neural networks.
- Mhaskar, Poggio (2016), Deep vs. Shallow Networks: an Approximation Theory Perspective.
- Poggio, Banburski, Liao (2019), Theoretical Issues in Deep Networks.
- Hestness et al. (2017), Deep Learning Scaling is Predictable, Empirically.
- Kaplan et al. (2020), Scaling Laws for Neural Language Models.
- Hoffmann et al. (2022), Training Compute-Optimal Large Language Models.

Part II — Generalization & Spectra

- Zhang et al. (2017), Understanding Deep Learning Requires Rethinking Generalization.
- Belkin, Hsu, Ma, Mandal (2019), Reconciling modern ML practice and the classical bias–variance trade-off.
- Nakkiran et al. (2019), Deep Double Descent.
- Belkin, Hsu, Xu (2020), Two models of double descent for weak features.
- Bartlett, Long, Lugosi, Tsigler (2020), Benign Overfitting in Linear Regression.
- Soudry et al. (2018), The Implicit Bias of Gradient Descent on Separable Data.
- Nacson et al. (2019), SGD on Separable Data.
- Jacot, Gabriel, Hongler (2018), Neural Tangent Kernel.

References — Part III, and spectral methods

Part III — The Virtue of Complexity

- Kelly, Malamud, Zhou (2024), The Virtue of Complexity in Return Prediction, *Journal of Finance*.
- Kelly, Malamud (2025), Understanding the Virtue of Complexity, SFI Research Paper 25-96.
- Goyal, Welch (2008), A Comprehensive Look at the Empirical Performance of Equity Premium Prediction, *RFS*.
- Rahimi, Recht (2007), Random Features for Large-Scale Kernel Machines, NeurIPS.

Spectral methods & overfitting taxonomy

- Mallinar et al. (2024), Benign, Tempered, or Catastrophic: A Taxonomy of Overfitting.
- Martin, Mahoney (2021), Implicit Self-Regularization in Deep Neural Networks.
- Martin, Hinrichs (2025), SETOL: A Semi-Empirical Theory of (Deep) Learning.